FLOWGUARD / TEAM HANDOFF

Work Completed and Project Transfer Guide

Transformer-Based Network Intrusion Detection System for Intelligent Multi-Class Cyber Attack Classification

Prepared for teammate collaboration in Codex | 19 October 2026

This document summarizes the software, experiments and submission materials created for the project. It explains how to transfer the complete folder, run the existing models on another computer and continue development in a new Codex project.

Transfer item	Included
Full project ZIP	Source, trained models, sampled data, results, tests and documents
Presentation	15-slide editable PowerPoint
Technical report	16-page PDF with methodology and measured results
Codex continuation	CODEX_HANDOFF.md with project map and ready-to-paste prompt
Website/dashboard	Local Streamlit application; no public hosted URL exists

The package is a runnable research prototype. Physical live capture and target-network reliability remain unverified. Research simulations and recorded data experiments should not be described as production deployments.

1. Software and research work completed

Area	Implemented work
Data pipeline	Pinned CIC-IDS2017 mirror downloads; source hashes/provenance; chunked audit; duplicate/conflict removal; explicit label mapping; sampled train/validation/test splits.
Preprocessing	Training-only median imputation, signed log transform and standardization; strict input schema and invalid-value checks.
Core classification	Five-class and seven-class trained feature-token Transformers. Seven labels include Benign, DoS, DDoS, Botnet, Port Scan, Brute Force and Infiltration.
Controlled comparisons	Logistic Regression, Random Forest, MLP, CNN, LSTM, GRU and Transformer on matching feature split rows.
Contextual architectures	Causal Temporal Transformer, TabTransformer, flow-grid ViT, scratch-trained BERT-style feature encoder and shared-endpoint flow GCN.
Adaptation	Three-client local FedAvg simulation; supervised prequential online updates; Page-Hinkley loss-change detector.
Explainability	Actual SHAP KernelExplainer and LIME Tabular integrations, plus median-replacement sensitivity.
Packet pipeline	Passive IPv4 TCP/UDP aggregation, bounded flow storage, PCAP replay, interface-capture path and JSONL prediction events.
User interface	Streamlit profile selector, samples/CSV upload, predictions/export, metrics, charts, Extended lab and explanation controls.
Handoff/submission	Saved checkpoints, preprocessors, data manifests, notebook, verification logs, technical report, editable PPT, viva guide, launcher and teammate setup instructions.

The data sources listed in the assignment are alternatives. CIC-IDS2017 was selected; models were not trained on all listed datasets. Custom Mininet/NS-3/SDN simulation is not included.

2. Recorded experiment results

Seven-class model	Accuracy	Macro-F1	Benign FPR
LogisticRegression	93.71%	0.8767	13.33%
RandomForest	99.23%	0.9588	0.89%
MLP	97.68%	0.9348	5.33%
CNN	98.62%	0.9539	2.22%
LSTM	94.75%	0.8946	13.56%
GRU	96.13%	0.9223	6.44%
Transformer	97.02%	0.9409	4.44%

Random Forest performed better than the feature Transformer in this fixed sampled benchmark. The results are retained as measured; Transformer superiority is not assumed. Infiltration has only five feature-benchmark test examples.

Context experiment	Accuracy	Macro-F1	Test outputs
BERTStyleEncoder	97.19%	0.9111	3526
TabTransformer	95.09%	0.8647	3526
FlowViT	95.26%	0.8892	3526
FlowGraphNetwork	89.17%	0.6782	3526
TemporalTransformer	93.88%	0.8003	196
FederatedFedAvg	95.63%	0.7984	3526
OnlineLearning	97.16%	0.9189	3526

Temporal outputs label endpoints of nonoverlapping eight-flow windows. They differ from individual-flow evaluation rows. FedAvg and online-learning rows also have different training/adaptation protocols. These rows are not a controlled architecture ranking.

3. Verification and limits to preserve

The recorded development run passed all 22 automated tests, with no skips. The log is results/VERIFICATION.txt. Checks cover strict inputs, training-only medians, vector split separation, saved checkpoint parity, dashboard profiles, temporal inference, actual SHAP, chronological window membership, graph edges, FedAvg arithmetic, drift response and bidirectional aggregation. These checks were recorded during project development; the teammate should rerun them locally.

All 16 technical-report pages and all 15 finalized slides were rendered and visually reviewed. The PPT passed structural, layout and editable-chart checks; native Microsoft PowerPoint opening was not verified. PCAP replay processed 48 synthetic packets into 12 completed prediction events. Synthetic replay is a plumbing test, not an attack-accuracy evaluation.

Boundary	What must not be overstated
Physical capture	Npcap is absent on the original machine. The interface code exists, but live-interface operation is not validated.
Extraction shift	The custom packet statistics are not proven CICFlowMeter-equivalent; deployment accuracy is unknown.
Temporal coverage	196 test endpoints, no infiltration endpoint, only six benign endpoints. Rare-class and false-alert evidence is weak.
Chronology	Mirror timestamps omit AM/PM. A documented daytime correction heuristic is used; original PCAP timing was not independently checked.
Generalization	Historical sampled data; hosts/campaigns can cross blocks; one fixed configuration/seed; no multi-dataset validation.
Adaptation	FedAvg is a local simulation without privacy/distributed guarantees. Online updates require true labels.
Explanations/novel attacks	Saved LIME fidelity is poor. Anomaly known-attack recall is weak. No causal explanation, zero-day reliability or new architecture claim.

4. Run the project on the teammate's computer

Extract the complete ZIP to a short path, for example C:\Projects\Transformer_NIDS. Install Python 3.12. Open PowerShell inside that extracted folder and run:

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\SETUP_PROJECT.ps1
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Setup installs dependencies into the local .venv. Internet is required for installation. Saved models and sampled data are already included, so no training or raw-source download is needed for the demonstration. Do not transfer the original developer's virtual environment.

Open the URL printed in the terminal, normally http://localhost:8501. Keep the terminal open. On subsequent launches, double-click OPEN_PROJECT.cmd. Select Extended seven classes, classify the included examples, then open Extended lab for temporal/alternative models and explanations.

Website link explained

http://127.0.0.1:8851 is the original computer's local server address while it is running. It cannot be shared as a remotely accessible website. On the teammate's computer, localhost refers to their own machine, and their server will usually print port 8501.

There is no public hosted website URL at present. A permanent shared link requires a separate hosting deployment. The supplied Streamlit application is the runnable website implementation; do not claim it has already been published.

Optional packet demonstration

```
.\.venv\Scripts\python.exe scripts/monitor.py --pcap
  data/samples/synthetic_pipeline_demo.pcap
```

Enter the replay command on one line. Physical interface capture requires the official Npcap driver on Windows, capture permissions and target-interface testing. The application observes traffic and does not block it.

5. Continue development in Codex

Open the extracted Transformer_NIDS folder as a project in the teammate's Codex. The original chat history does not transfer. The handoff note, code, saved models and evidence files provide the project context.

Ready-to-paste starting prompt

Continue this existing FlowGuard project. Read CODEX_HANDOFF.md, README.md, docs/REQUIREMENT_TRACEABILITY.md and docs/COMPLETE_PROJECT_GUIDE.md first. Inspect existing code, checkpoints, data manifests and results before editing. Set up the local environment, run the dashboard and verify the existing tests. Preserve measured results and distinguish feature classification, chronological experiments, simulations and unverified deployment claims. Use separate output folders for new experiments. Then help implement my next requested change.

Start here	Use
app.py / nids/	Dashboard, model architectures, inference, capture and explanation code
scripts/	Acquisition, preparation, training, prediction, replay and document generation
artifacts/ / data/ / results/	Saved reference checkpoints, delivered splits, provenance and measured evidence
CODEX_HANDOFF.md	Portable commands, file map, findings, limits and continuation prompt
docs/	Report, PPT, summary PDF, requirement traceability, architecture and viva notes

Recommended next work

First reproduce the supplied demo and tests locally. Then validate live capture/extraction parity, collect more rare attack sequences, test stronger host/campaign and external-data holdouts, evaluate realistic client heterogeneity and drift, and perform broader controlled model tuning. Public hosting is a separate task if the team wants a shared website URL.

Personalize the report and PPT with team names, registration numbers, institution and guide. Keep source attribution and measured limitations. The ZIP excludes raw-source datasets, Python environments and packet-capture drivers; download scripts and provenance are included.